

Guía de Aprendizaje: Funciones para Leer Archivos

Malak Sanchez

Workshop 2026-II

21 de agosto de 2026

1. Motivación

Los datos dentro de un archivo de texto pueden estar distribuidos de diferentes maneras (caracteres repetidos, líneas de configuración, campos separados de datos), exigiendo distintas maneras especializadas que nos permitan importarlos con alta precisión en C.

2. Idea Fundamental

Las funciones `fgetc`, `fgets` y `fscanf` leen desde la posición actual del archivo y avanzan su cursor. Cada una permite procesar caracteres, líneas o datos con formato, respectivamente.

3. Sintaxis Básica

A continuación, una tabla comparativa con las principales funciones para extraer texto:

Función	¿Qué lee?	Caso de uso ideal
<code>fgetc</code>	Un solo carácter a la vez.	Contar letras específicas o parseo carácter por carácter.
<code>fgets</code>	Una cadena de texto (línea) hasta un salto de línea o límite.	Leer líneas completas de forma segura en archivos de configuración o <i>logs</i> .
<code>fscanf</code>	Datos formateados con un patrón fijo, ignorando espacios en blanco.	Leer registros tabulares, bases de datos simples u organizar numéricos.

```
int char_read = fgetc(file_ptr);
char *text_read = fgets(buffer, limited_size, file_ptr);
int data_matched = fscanf(file_ptr, "%d %s", &int_var, str_var);
```

4. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv){
4     if(argc < 2){
5         printf("Usage: %s <filename>\n", argv[0]);
6         return 1;
7     }
8     FILE *file = fopen(argv[1], "r");
9     if(file == NULL) return 1;
10
11     char buffer[100];
12
13     if(fgets(buffer, sizeof(buffer), file) != NULL){
14         printf("Primera linea: %s", buffer);
15     }
16
17     fclose(file);
18     return 0;
19 }
```

5. Explicación Paso a Paso

1. Verificamos que el usuario haya proporcionado un nombre de archivo al ejecutar el programa.
2. Accedemos a la primera posición del vector de argumentos `argv[1]`.
3. Abrimos el archivo en modo lectura con `fopen`.
4. Declaramos un arreglo local de caracteres que funcionará como búfer.
5. `fgets` lee como máximo `sizeof(buffer) - 1` caracteres y añade el terminador nulo, evitando desbordar el búfer.
6. Si la lectura es correcta, mostramos la primera línea y cerramos el archivo con `fclose`.

6. Qué ocurre en memoria

La biblioteca de C mantiene un flujo `FILE` con información como la posición actual y un búfer interno. Cada llamada de lectura obtiene datos desde esa posición y avanza el cursor del archivo.

7. Patrones comunes de uso

7.1. Lectura secuencial

Para recorrer un archivo carácter por carácter, se lee hasta que `fgetc` devuelve EOF:

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv){
4     if(argc < 2){
5         printf("Usage: %s <filename>\n", argv[0]);
6         return 1;
7     }
8     FILE *file = fopen(argv[1], "r");
9     if (file == NULL) return 1;
10
11     int current_char;
12     while ((current_char = fgetc(file)) != EOF) {
13         printf("%c", (char)current_char);
14     }
15
16     fclose(file);
17     return 0;
18 }
```

8. Medir un archivo manipulando el cursor dinámicamente

Cuando no sabemos la cantidad de elementos o bytes totales que nos esperan dentro del archivo, requerimos manipular el cursor interno usando la función `fseek`. Esta función se apoya en anclas especiales como `SEEK_END` (el final absoluto del archivo) y `SEEK_SET` (el comienzo del mismo).

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char **argv){
5     if(argc < 2){
6         printf("Usage: %s <filename>\n", argv[0]);
7         return 1;
8     }
9     FILE *file = fopen(argv[1], "r");
10    if (file == NULL) return 1;
11
12    // 1. Movemos el cursor al extremo final del archivo
13    fseek(file, 0, SEEK_END);
14
15    // 2. Averiguamos la posicion actual, que equivale al total
16    //    de bytes
17    long file_size = ftell(file);
18    printf("File size: %ld bytes\n", file_size);
19 }
```

```
19 // 3. Regresamos el cursor al principio para reanudar la
    lectura
20 fseek(file, 0, SEEK_SET);
21
22 // Aqui ya podriamos usar 'file_size' de forma segura
23 // en un malloc para contener todos los datos a la vez si
    fuese util.
24
25 fclose(file);
26 return 0;
27 }
```

9. Errores Comunes

- **No comprobar el final o un error de lectura:** Un ciclo con `fgetc` debe terminar al recibir EOF; `fgets` debe comprobarse contra NULL.
- **Ignorar el valor de retorno de `fscanf`:** La función indica cuántas conversiones realizó; ese valor debe validarse antes de usar los datos.

10. Buenas Prácticas

- Use `fgets` para procesar líneas de texto y `fscanf` solo cuando el formato de entrada sea conocido.
- Valide siempre el valor de retorno de las funciones de lectura.

11. Ejercicios

1. Escriba una rutina con `fgetc` que cuente el porcentaje de apariciones de la vocal A en un archivo de texto.
2. Use `fscanf` para leer identificación, curso y calificación de cada registro, y calcule el promedio por curso.